[Back to index]

Query Evaluation Techniques for Large Databases (1-4)

Goetz Graefe

Summary by: Steve Gribble and Armando Fox

One-line summary: An encyclopoedic survey of query evaluation techniques - sorting, hashing, disk access, and aggregation/duplicate removal are dealt with in these sections (1-4) of the paper.

Overview/Main Points

- Steps to process a query: parsing, validation, resolution, optimization, plan compilation, execution. This paper focuses on the last step (execution).
- **Architecture of query engines**:
  - Query processing algorithms iterate over members of input sets; algorithms are algebra operators. The *physical algebra* is the set of operators, data representations, and associated cost functions that the database execution engine supports, while the *logical algebra* is more related to the data model and expressible queries of the data model (e.g. SQL).
  - Synchronization and transfer between operators is key. Naive methods include creation of temporary files/buffers, or using one process per operator and use IPC. Practical method is to implement all operators as a set of procedures (open, next, and close), and have operators schedule each other within a single process via simple function calls. Each time an operator needs another piece of data ("granule"), it calls its data input operator's *next* function to produce one. Operators structured in such a manner are called *iterators*.
  - Query plans are algebra expressions and can be represented as trees. Left-deep (every right subtree is a leaf), right-deep (every left-subtree is a leaf), and bushy (arbitrary) are the three common structures. In a left-deep tree, each operator draws input from one input, and an inner loop interates over the other input.
- **Sorting**:
  - All sorting in "real" database systems use merging techniques, since very large data sets are expected. Sorting modules' interfaces should follow the structure of iterators.
  - Exploit duality of quicksort and mergesort. Sort proceeds in *divide* phase and *combine* phase. One of the two phases is based on logical keys (indexes), the physically arranges data items (which phase is logical is particular to an algorithm). Two subalgorithms: one for sorting a run within main memory, another for managing runs on disk or tape. Degree of fan-in (number of runs merged in a given step) is a key parameter.
  - Quicksort and replacement-selection are the two algorithms of choice for creating the set of initial (level-0) runs. Replacement selection fills memory in a priority heap, smallest key is written to a run and replaced from next in input; this replacement will probably be bigger than the just written item, so we can then iterate. If not, put mark replacement for next run file. Quicksort has bursty I/O pattern, RS smoothly alternates between read and write operations.
  - Level-0 runs are merged into level-1 runs. Buffer space must be dedicated to each input run and the merge output; unit of I/O is a *cluster*. Efficiency concerns are:
    1. scans are faster if read-ahead/write-behind are used.
    2. large cluster sizes for run files is beneficial. Determine optimal either by physical disk characteristics, or by matching processor and I/O latencies.
    3. the number of runs is usually not a power of the fan-in F, and therefore some merges proceed with less than F inputs. Optimizations can take place.
    4. Some operators require multiple sorted inputs, so memory must be divided among multiple final merges; this final fan-in and the normal fan-in of each sort should be specified separately.
- **Hashing**:
  - Hashing should be consider for equality matches, in general.

    - Hashing-based query processing algos use in-memory hash table of database objects; if data in hash table is bigger than main memory (common case), then *hash table overflow* occurs. Three techniques for overflow handling exist:
        - Avoidance: input set is partitioned into F files before any in-memory hash table is built. Partitions can be dealt with independently. Partition sizes must be chosen well, or recursive partitioning will be needed.
        - Resolution: assume overflow won't occur; if it does, partition dynamically.
        - Hybrid: like resolution, but when partition, only write one partition to disk, keep the rest in memory.
    - Assigning hash buckets to partitions needs to be optimized so that disk accesses result in clustered buckets both physically and logically. Three ways to assign hash buckets to partitions. First, each time a hash overflow occurs, fixed number of hash buckets assigned to a new output partition; i.e. fan out set a-priori. Second (bucket tuning/dynamic destaging) a large number of small partition files is collapsed into fewer files no larger than main memory. Third, statistics gathering before hybrid hashing commences is used.
    - Quality of hash function is key, so you don't end up with skewed recursive partition depths, leading to poor performance and much disk I/O.
- **Disk Access**:
    - File scans can be made fast with read-ahead (track-at-a-crack). Requires contiguous file allocation, so may need to bypass OS/file system.
    - **Indices**:
        - B-trees, B+-trees, B*-trees are your friends. So are hashing structures. We've seen this in other papers, so I won't go into it here.
        - Leafs of indices should point to data records, not contain them, so one can separate record lookup from index scanning. Advantages:
            - it is possible to scan an index without ever retrieving records, i.e. if only salary values needed and the index is on salary.
            - even if none of the indices is sufficient by itself, multiple indices can be joined to satisfy query requirements.
            - if two or more indicies apply to individual clauses of a query, take union/intersection of two index scans.
            - joining two tables can be done by joining indices on the two join attributes and then doing record retrievals in underlying data sets.
    - **Buffer management**:
        - Cache data in I/O buffer. LRU is wrong for many database operations. Buffer mgmt mechanisms usually provide fix/unfix semantics on a buffer page, which iterator implementations can take advantage of when passing buffer pages amongst themselves.
- **Aggregation/duplicate removal**:
    - *Scalar aggregate* calculates a single scalar value from a unary input relation, can be computed in single pass over data. *Aggregate functions* determine set of values from a binary input relation, e.g. sum of salaries from each department ("group-by" semantics).
    - Aggregate functions require grouping; this grouping is virtually identical to what is required in duplicate removal, so aggration and duplicate removal are treated as the same.
    - **Nested loop aggregation**: naive method: using temporary file to accumulate output, loop for each input item over output file, and either aggregate in the new item into appropriate output item, or append new output item to file. Inefficient for large data sets.
    - **Aggregation based on sorting**: sorting brings items together. Use to detect and remove duplicates or do aggregation as soon as possible; implement in routines that create the run files. "Early aggregation".
    - **Aggregation based on hashing**: if hash function puts items of same group in same bucket (or nearby bucket), duplicate removal/aggregatation can be done when inserting into hash table.
    - Early aggregation is a big win, performance-wise, as size of run files or hash table is reduced. Both sort and hash result in logarithmic performance based on input data set size.
    - Multilevel aggregation? Difficult to avoid multiple joins.
    - Some applications don't require exact aggregation functions; approximations that are more efficient will do (e.g. real-time systems' trade off between precision and response time).

Relevance

An encyclopoedic survey of techniques, with good commentary explaining relevance and comparing relative merits of all of the techniques. Overall, an excellent paper.

Flaws

- Lots of effort is spent deriving cost equations, but many assumptions are made in the derivations that may make the final equations somewhat unrealistic. Also, are the costs worst-case, best-case, or average case in general?
- The paper always talks about "large data sets" as a motivation for some of the techniques, but exact or specific semantics of the large data sets may dictate how techniques will perform in reality - e.g. multimedia video data v.s. human genome data v.s. multi-dimensional geographic data probably all have individual optimizations that affect these techniques, but such possible optimizations are not discussed.

Back to index